

Documentação Técnica

Guia completo do código-fonte — pasta a pasta, ficheiro a ficheiro

Gerado em 20 de agosto de 2026 · Cobre as Fases 1 a 4 do projeto
Repositório: github.com/Erick0011/Ponto24academico · Branch: fase-3-sistema-pastas

Este documento foi escrito para apoiar debugging e planeamento de melhorias futuras. Descreve o que cada módulo faz, as suas funções/classes principais e as decisões de arquitetura por trás do código — não substitui a leitura do código-fonte em si, mas serve de mapa para o encontrar rapidamente.

Sumário

Visão geral

- O que é o projeto
- Stack tecnológica
- Arquitetura geral
- Árvore de pastas completa

O coração da app — `app/__init__.py`

Modelos — `app/models/`

Rotas — `app/routes/`

Serviços — `app/services/`

Utilitários — `app/utils/`

Templates — `app/templates/`

Estáticos — `app/static/`

Configuração — `config/`

Migrações — `migrations/`

Ficheiros de raiz — `manage.py`, `run.py`, `wsgi.py`

Convenções e padrões de arquitetura

Modelo de dados — tabelas e relações

Fluxos ponta-a-ponta importantes

Guia rápido de debugging

1. Visão geral

1.1 O que é o projeto

O Ponto 24 Académico é uma plataforma web onde estudantes angolanos partilham e descobrem materiais académicos (provas, resumos, exercícios, apontamentos, gabaritos), organizados por instituição, curso, disciplina e ano. Além do acervo de materiais, a plataforma tem uma secção de Comunidade estilo fórum, gamificação por créditos, um painel administrativo completo e um sistema de email transaccional e de marketing em massa.

O projeto foi construído em fases incrementais, cada uma isolada na sua própria branch git (sequencialmente ramificadas): segurança e limpeza (Fase 1), painel admin profissional e candidaturas (Fase 2), sistema de pastas (Fase 3) e Comunidade (Fase 4). Este documento descreve o estado do código no final da Fase 4.

1.2 Stack tecnológica

Camada	Tecnologia	Onde entra no código
Framework web	Flask 3	<code>app/__init__.py</code> , todos os blueprints
ORM	Flask-SQLAlchemy	<code>app/models/*.py</code>
Migrações	Flask-Migrate (Alembic)	<code>migrations/</code>
Autenticação	Flask-Login	<code>app/routes/auth.py</code> , <code>User.is_authenticated</code>
CSRF	Flask-WTF (CSRFProtect)	<code>csrf_token()</code> em todos os formulários POST
Rate limiting	Flask-Limiter	decorador <code>@limiter.limit(...)</code> em rotas públicas
Base de dados	SQLite (dev) / PostgreSQL (prod)	<code>DATABASE_URL</code> em <code>config/settings.py</code>
Armazenamento de ficheiros	Disco local ou Cloudflare R2 (S3-compatible)	<code>app/services/upload_service.py</code> , <code>r2_service.py</code>
Imagens	Pillow	geração de thumbnails 300×300
Email	<code>smtplib</code> puro (sem Flask-Mail)	<code>app/services/mail_service.py</code>
Frontend	Bootstrap 5.3 + Bootstrap Icons (CDN) + CSS próprio	<code>app/static/css/ponto24.css</code>
Produção	Gunicorn	<code>wsgi.py</code>

1.3 Arquitetura geral

A app segue o padrão **application factory**: em vez de um objeto `Flask` global criado no topo de um ficheiro, existe uma função `create_app()` que constrói e devolve a instância — isto permite criar múltiplas instâncias configuradas de forma diferente (por exemplo, uma para testes com base de dados em memória).

O ciclo de vida de um pedido HTTP típico segue este caminho:

1. `run.py` ou `wsgi.py` chama `create_app()` , que devolve a app Flask já configurada
2. O pedido entra, passa pelos `before_request` hooks registados em `app/__init__.py` (bloqueio de pré-lançamento, verificação de confirmação de email)
3. O Flask despacha para a rota certa dentro de um dos 6 blueprints
4. A rota faz pouco por si — delega a lógica de negócio para uma função em `app/services/`
5. O serviço interage com os modelos (`app/models/`) via SQLAlchemy
6. A rota devolve um `render_template(...)` (Jinja2, em `app/templates/`) ou um `redirect()`

7. O `after_request` hook acrescenta cabeçalhos de segurança à resposta antes de sair

1.4 Árvore de pastas completa

```
Ponto24academico/
├── app/
│   ├── __init__.py           Application factory
│   ├── models/              9 ficheiros – um por domínio
│   ├── routes/              6 blueprints
│   ├── services/            10 ficheiros – lógica de negócio
│   ├── utils/               honeypot.py, validacao.py
│   ├── static/
│   │   ├── css/ponto24.css   Design system próprio
│   │   ├── img/
│   │   └── uploads/          Ficheiros enviados (só modo disco local)
│   └── templates/           ~70 ficheiros .html (Jinja2)
│       ├── admin/, auth/, comunidade/, dashboard/, email/,
│       ├── errors/, main/, materials/, notificacoes/, partials/
│       └── base.html, index.html
├── config/
│   ├── __init__.py
│   └── settings.py           Config, DevelopmentConfig, ProductionConfig, TestingConfig
├── migrations/              Alembic – 1 ficheiro por migração (14 no total)
├── docs/
│   ├── TESTES.md            Checklist manual (200+ testes)
│   └── PONT024_DOCUMENTACAO_TECNICA.pdf Este documento
├── logs/                   ponto24.log (rotativo, criado em runtime)
├── manage.py                Comandos CLI
├── run.py                   Entrada em desenvolvimento
├── wsgi.py                  Entrada para Gunicorn
├── requirements.txt
├── .env.example
└── README.md
```

2. O coração da app — `app/__init__.py`

Este é o ficheiro mais importante do projeto para perceber "como tudo se liga". A função `create_app(config_name=None)` faz, por esta ordem:

1. **Carrega a configuração** — lê `FLASK_ENV` (ou o argumento) e aplica a classe certa de `config/settings.py`
2. **Prepara logging** — cria `logs/` se não existir, regista um `RotatingFileHandler` (10 MB × 5 ficheiros)
3. **Inicializa extensões** — `db` (SQLAlchemy), `migrate`, `csrf`, `limiter`, `login_manager`
4. **Importa todos os modelos** explicitamente (com `# noqa: F401`) — necessário para que o Alembic os "veja" ao gerar migrações, mesmo que nenhuma rota os importe diretamente
5. **Regista os 6 blueprints**
6. **Regista dois `before_request` hooks:**
 - `bloquear_pre_lancamento()` — se a configuração `modo_pre_lancamento` estiver ativa e o visitante não tiver sessão, redireciona para a lista de espera (exceto numa whitelist de endpoints públicos)
 - `verificar_email_confirmado()` — utilizadores autenticados com email por confirmar há mais de 7 dias são redirecionados para `auth.email_nao_confirmado`
7. **Regista um `context_processor`** (`inject_notif_count`) que injeta em *todos* os templates: contagem de notificações não lidas, dias restantes para confirmar o email, configurações públicas (`cfg`), o banner de anúncio ativo, e duas funções auxiliares de URL de imagem (`thumb_url`, `comunidade_imagem_url`)
8. **Regista o Jinja global `paginate_url(page)`** — constrói a URL da página N preservando todos os parâmetros de query atuais (usado em toda a paginação do site)
9. **Regista o `after_request`** que acrescenta cabeçalhos de segurança a toda resposta
10. **Regista os error handlers** 403/404/429/500

Onde procurar primeiro quando algo "estranho" acontece em todas as páginas (um redirecionamento inesperado, uma variável em falta num template, um cabeçalho HTTP): é quase sempre aqui, nos hooks e no context processor.

3. Modelos — `app/models/`

Todos os modelos herdam de `db.Model` (Flask-SQLAlchemy). Convenção do projeto: nomes de tabelas e colunas em português, timestamps em UTC via `datetime.utcnow` como `default`. `app/models/__init__.py` reexporta todas as classes para que possam ser importadas como `from app.models import X`.

3.1 `user.py` — `User`

`app/models/user.py`

Representa um utilizador. Hierarquia de permissões: `is_admin` > `is_moderador` > utilizador normal. Guarda o saldo de créditos (`creditos`), contadores (`total_uploads`, `total_downloads`), estado de conta (`is_active`), verificação de email (`email_verificado`) e opt-out de marketing (`aceita_marketing`, adicionado na Fase 4).

Métodos-chave:

- `set_password()` / `check_password()` — hashing via `werkzeug.security`
- `ganhar_creditos()` / `gastar_creditos()` / `tem_creditos()` — a única forma correta de alterar o saldo (nunca atribuir `user.creditos = x` diretamente fora do `creditos_service`)
- `nivel` (property) — deriva o nível de gamificação (Novato/Bronze/Prata/Ouro/Expert) a partir do saldo de créditos, devolve um dicionário pronto a usar no template (nome, ícone, cor, percentagem até ao próximo nível)

3.2 `material.py` — `Categoria`, `Material`, `Favorito`, `Avaliacao`

`app/models/material.py`

Categoria — tipo de material (Prova, Resumo, ...), CRUD simples pelo admin.

Material — o modelo central da plataforma. Estados de moderação como constantes de classe: `STATUS_PENDENTE`, `STATUS_APROVADO`, `STATUS_REJEITADO`. Campos de classificação (`instituicao`, `curso`, `disciplina`, `ano_letivo`, `ano_escolar`, `semestre`) são **texto livre**, sem tabelas de referência — não há normalização aqui de propósito, para não travar o utilizador a uma lista fechada de instituições/cursos. `ficheiro_hash` (SHA-256) permite detetar duplicados exatos. `grupo_upload` (UUID) agrupa várias páginas do mesmo documento submetidas de uma vez — um "material" no sentido do utilizador pode ser várias linhas na tabela.

Properties úteis: `esta_aprovado`, `titulo_base` (remove o sufixo "— Pág. X de N" dos grupos), `e_imagem`, `tem_preview`, `thumbnail_path`.

Favorito — toggle boolean (linha existe = favoritado), com `UniqueConstraint(utilizador_id, material_id)`.

Avaliacao — 1 a 5 estrelas + comentário opcional, um por (utilizador, material) — é o precedente de UX que a Comunidade reaproveitou para o formulário de resposta (upsert por unique constraint).

3.3 pasta.py — Pasta Fase 3

`app/models/pasta.py`

Árvore auto-referenciada (`parent_id` aponta para `pastas.id`) usando a técnica de **caminho materializado**: a coluna `caminho` guarda os ids dos antepassados separados por `/` (ex: `"3/7/12"`). Isto torna a busca de todos os descendentes de uma pasta uma simples query `LIKE 'caminho/%'`, em vez de uma CTE recursiva. Esta técnica só é segura porque o escopo do produto nunca precisa de mover uma pasta já criada para outro sítio da árvore (só criar e eliminar) — se isso mudar no futuro, o `caminho` de todos os descendentes teria de ser recalculado.

`ids_subquery()` devolve os ids da própria pasta + todos os descendentes — usado para filtrar materiais por pasta, incluindo os das subpastas.

3.4 anuncio.py — Anuncio

`app/models/anuncio.py`

Banners publicitários pagos, com rotação por percentagem (um anunciante compra "10% do espaço", "25%", etc.). `Anuncio.selecionar()` faz uma seleção aleatória ponderada entre os anúncios ativos hoje. Desde a Fase 4, o resultado desta seleção já não é mostrado em todo o site — só dentro do feed da Comunidade (ver secção 12).

3.5 candidaturas.py — Candidatura

`app/models/candidaturas.py`

Candidaturas de voluntários/colaboradores (não confundir com registo de conta normal) — tem campos ricos (motivação, experiência, disponibilidade para liderança). `contribuicoes` é guardado como JSON-string dentro de um `Text` (não há tabela própria para isto).

3.6 lista_espera.py — ListaEspera, RelatorioMaterial

`app/models/lista_espera.py`

ListaEspera — inscrições de acesso antecipado, usadas quando `modo_pre_lancamento` está ativo.

RelatorioMaterial — denúncias de materiais, com `MOTIVOS` como lista de tuplos e `status` pendente/resolvido/ignorado. Este é o molde que a Fase 4 copiou para `ComunidadeRelatorio`.

3.7 `configuracao.py` — `Configuracao`

`app/models/configuracao.py`

Tabela chave-valor simples para configurações editáveis em runtime pelo admin (contactos, redes sociais, modo pré-lançamento) sem precisar de alterar variáveis de ambiente e reiniciar a app.

3.8 `notificacao.py` — `Notificacao`

`app/models/notificacao.py`

Notificações in-app (não emails). `tipo` é uma string livre com constantes de classe (`TIPO_MATERIAL_APROVADO` , `TIPO_COMUNIDADE_RESPOSTA` , ...), sem enum/constraint na base de dados — acrescentar um novo tipo nunca exige migração.

3.9 `kpi.py` — `PesquisaLog`

`app/models/kpi.py`

Um registo por termo de pesquisa único visitado na página 1 dos resultados — usado só para métricas no painel de KPI.

3.10 `atividade.py` — `AtividadeLog` Fase 2

`app/models/atividade.py`

O log de auditoria da plataforma. Cada evento é uma string-constante de classe agrupada por domínio (materiais, pastas, candidaturas, utilizadores, autenticação, marketing, comunidade). Colunas `alvo_tipo` (string livre, ex: `"material"` , `"comunidade_post"`) e `alvo_id` (inteiro, sem FK) formam uma **referência polimórfica por convenção** — o mesmo padrão reaparece em `RelatorioMaterial` e em `ComunidadeVoto / ComunidadeRelatorio` . Evita ter uma tabela de log por tipo de entidade, ao custo de o SGBD não conseguir garantir a integridade referencial destes campos — a validade de `alvo_id` é responsabilidade do código, não da base de dados.

3.11 `campanha_email.py` — `CampanhaEmail` , `CampanhaEmailDestinatario` Fase 4

`app/models/campanha_email.py`

`CampanhaEmail` guarda uma campanha de marketing (assunto, corpo HTML, público-alvo, estado, contadores de enviados/falhados, contador diário para o limite de segurança). `CampanhaEmailDestinatario` é uma linha por destinatário — a decisão de arquitetura mais importante

aqui: em vez de simplesmente iterar uma lista de utilizadores e enviar, cada destinatário fica registado na base de dados com o seu próprio estado (`pendente` / `enviado` / `falhado`). Isto torna o envio **retomável** — se o processo for interrompido (reinício do servidor, limite diário atingido), a próxima execução processa só quem ainda está `pendente` , sem duplicar envios.

3.12 `comunidade.py` — `ComunidadePost` , `ComunidadePostImagem` , `ComunidadeResposta` , `ComunidadeVoto` , `ComunidadeRelatorio` Fase 4

`app/models/comunidade.py`

`ComunidadePost` — publicação com `tipo` (dúvida/discussão/aviso), `fixado` (destaque pelo admin), e dois contadores denormalizados mantidos por código, não recalculados a cada leitura: `respostas_count` e `votos_score` .

`ComunidadeVoto` é o modelo mais interessante desta secção: post e resposta partilham a *mesma* tabela de votos, através de `alvo_tipo` (`"post"` / `"resposta"`) + `alvo_id` — o mesmo padrão polimórfico-por-convenção do `AtividadeLog` . Um `UniqueConstraint(utilizador_id, alvo_tipo, alvo_id)` garante um voto por utilizador por alvo; o valor (`+1` / `-1`) é o que muda quando o utilizador troca de opinião. A lógica de aplicar/trocar/remover um voto vive em `comunidade_service.aplicar_voto()` (secção 5.9), não no modelo.

`ComunidadeRelatorio` — cópia do padrão de `RelatorioMaterial` , também com `alvo_tipo` / `alvo_id` .

4. Rotas — `app/routes/`

Seis blueprints. Convenção geral: as funções de rota fazem validação de input, chamam serviços, e devolvem `render_template` / `redirect`. A lógica pesada não vive aqui.

4.1 `auth.py` — `/auth`

`app/routes/auth.py` · 269 linhas

Registo, login, logout, confirmação de email por token, recuperação de palavra-passe.

Rota	O que faz
POST <code>/auth/registar</code>	Cria conta; em modo pré-lançamento exige token de convite válido cujo email coincida com o do formulário; atribui créditos iniciais; faz login automático; dispara email de boas-vindas + email de confirmação
POST <code>/auth/entrar</code>	Login; regista tentativas falhadas no <code>AtividadeLog</code> (incluindo motivo: credenciais inválidas vs. conta suspensa); protege contra <i>open redirect</i> no parâmetro <code>next</code> (rejeita se tiver domínio externo)
GET <code>/auth/confirmar/<token></code>	Valida o token (24h) e marca <code>email_verificado=True</code>
GET/POST <code>/auth/recuperar</code> , <code>/auth/redefinir/<token></code>	Fluxo de recuperação de senha (token de 1h); resposta idêntica quer o email exista ou não, para não permitir enumeração de contas

4.2 `main.py` — `/`

`app/routes/main.py` · 513 linhas

Página inicial, dashboard, perfil, páginas institucionais, e vários formulários públicos.

Rota	O que faz
GET /	Landing page com estatísticas públicas (contagens reais, não fixas) e materiais recentes/populares; redireciona para o dashboard se autenticado
GET /dashboard, /perfil, /utilizador/<id>	Área privada + perfil público de qualquer utilizador
GET/POST /suporte	Formulário de contacto — envia email para o endereço de suporte configurado
GET/POST /juntar-se	Formulário de candidatura a colaborador (cria <code>Candidatura</code>)
GET/POST /acesso-antecipado	Lista de espera
GET /anunciar	Página de vendas de espaço publicitário — mostra <code>Anuncio.percentagem_disponivel()</code>
GET /robots.txt, /sitemap.xml	SEO — sitemap inclui todos os materiais aprovados
GET/POST /marketing/cancelar/<token>	Cancelamento público de subscrição de marketing, sem exigir login

Fase 4

4.3 materiais.py — /materiais

app/routes/materiais.py · 584 linhas

O núcleo funcional original da plataforma.

Rota	O que faz
GET /materiais/	listar() — pesquisa com filtros combináveis (categoria, instituição, disciplina, ano, pasta, texto livre) + paginação; usa o helper de módulo _grupo_metadados() para pré-carregar contagens/thumbnails de grupos sem N+1 queries
GET /materiais/ pastas, /materiais/ pastas/<id> Fase 3	Navegação pública pela árvore de pastas — reaproveita _grupo_metadados() e o mesmo template de card
GET/POST /materiais/ submeter	Upload — aceita múltiplos ficheiros de uma vez (cria um grupo_upload se mais que um), calcula slug da categoria para o subfolder de armazenamento, deteta duplicados por hash antes de persistir, notifica todos os moderadores do novo pendente
GET /materiais/<id>/ preview, /download	Preview não cobra créditos; download cobra (exceto ao próprio autor) via cobrar_creditos_download(); ambos servem o ficheiro local ou geram uma presigned_url do R2
POST /materiais/<id>/ guardar	Toggle de favorito (JSON, chamado via JS)
POST /materiais/<id>/ avaliar	Upsert de avaliação (1-5 estrelas + comentário), bloqueia auto-avaliação, notifica o autor
POST /materiais/<id>/ reportar	Cria RelatorioMaterial, bloqueia denúncias duplicadas pendentes do mesmo utilizador

4.4 comunidade.py — /comunidade Fase 4

app/routes/comunidade.py · 277 linhas

Rota	O que faz
GET /comunidade/	<code>feed()</code> — paginado, filtrável por <code>tipo</code> , ordenável por <code>recentes</code> / <code>votados</code> (posts fixados sempre primeiro)
GET/POST /comunidade/novo	Cria post; até 6 fotos (extensões inválidas são ignoradas sem bloquear o resto); honeypot + rate limit <code>10/hora</code> ; créditos + log de auditoria
GET /comunidade/<id>	<code>detalhe()</code> — post + respostas paginadas, pré-carrega o voto do utilizador atual para cada resposta visível (evita N+1 no template, mas ainda é uma query por resposta — ver secção 15 para uma possível otimização)
POST /comunidade/<id>/responder	Rate limit <code>30/hora</code> ; incrementa <code>respostas_count</code> ; notifica o autor do post
POST /comunidade/<id>/votar, /respostas/<id>/votar	Bloqueia auto-voto; delega a <code>comunidade_service.aplicar_voto()</code> ; redireciona para <code>request.referrer</code> (mantém o utilizador onde estava — feed ou detalhe)
POST /comunidade/<id>/denunciar, /respostas/<id>/denunciar	Cria <code>ComunidadeRelatorio</code> , mesmo padrão de dedup do <code>reportar()</code> de materiais
POST /comunidade/<id>/eliminar, /respostas/<id>/eliminar	Autor ou moderador; <code>eliminar_post_interno()</code> é uma função module-level reaproveitada também pelo admin (ver 4.5) para não duplicar a lógica de apagar imagens do disco/R2

4.5 admin.py — /admin

app/routes/admin.py · 1142 linhas — o maior ficheiro do projeto

Todas as rotas exigem `@login_required` mais um de dois decoradores definidos no topo do ficheiro: `admin_required` (só `is_admin`) ou `moderador_required` (`is_admin` OU `is_moderador`). Organizado em secções claramente comentadas:

Secção	Resumo
Painel	<code>panel()</code> — contagens rápidas para o dashboard admin
KPI	Métricas agregadas + atividade recente do <code>AtividadeLog</code>
Moderação de materiais	<code>moderacao()</code> , <code>aprovar()</code> , <code>rejeitar()</code> , <code>rever_material()</code> (revisor com navegação prev/next pela fila)
Todos os materiais	Listagem admin com eliminar e mover para pasta
Utilizadores	Toggle admin/moderador/ativo, ajuste manual de créditos
Categorias	CRUD
Pastas Fase 3	Criar/eliminar pastas, mover materiais (grupo inteiro) para uma pasta
Configurações	Editor da tabela <code>Configuracao</code>
Lista de espera	Convidar (gera token + envia email) / eliminar
Candidaturas Fase 2/4	Aprovar/rejeitar — desde a Fase 4, dispara automaticamente o email correspondente
Relatórios de materiais	Fila de denúncias, resolver/ignorar
Logs	Visualização do ficheiro de log da app
Anúncios	CRUD de banners (upload local ou R2)
Marketing Fase 4	Criar/ver/enviar/cancelar campanhas de email em massa (ver secção 5.8)
Comunidade Fase 4	Fila de denúncias, eliminar conteúdo denunciado, fixar/desafixar posts

4.6 `notificacoes.py` — `/notificacoes`

`app/routes/notificacoes.py` · 56 linhas

Listagem paginada (marca tudo como lido ao visitar), marcar uma como lida + redirecionar para o seu `url`, marcar todas como lidas, e um endpoint JSON de contagem (pensado para polling futuro no frontend, atualmente não usado por JS nenhum).

5. Serviços — `app/services/`

Convenção importante e consistente em quase todos os serviços de "criar registo": a função **adiciona à sessão mas não faz commit** — o commit é sempre responsabilidade do código que chamou a rota, para poder agrupar várias alterações (ex: criar material + registar atividade + notificar) numa única transação atómica. As exceções deliberadas a isto são os serviços de *créditos*, que fazem o próprio commit (ver 5.3).

5.1 `upload_service.py`

`app/services/upload_service.py` · 218 linhas

`guardar_ficheiro(file_obj, subfolder="")` é a função central: valida a extensão contra `ALLOWED_EXTENSIONS`, gera um nome único (UUID), calcula SHA-256, grava em disco local ou R2 consoante `_usar_r2()` (testa se `R2_ENDPOINT` está definido), e gera automaticamente uma thumbnail 300×300 para imagens. É completamente genérica — não sabe nada sobre "Material", por isso é reaproveitada tal-e-qual pelos posts da Comunidade (Fase 4), só muda o `subfolder` passado.

Segunda metade do ficheiro (acrescentada na Fase 3): primitivas para **mover** ficheiros já guardados para uma nova pasta. `mover_ficheiro()` nunca toca na base de dados — só move o ficheiro físico e devolve o novo caminho; quem chama decide se atualiza a BD. `mover_grupo_para_pasta()` é **all-or-nothing**: só devolve `ok=True` se todos os ficheiros do grupo moverem com sucesso; em falha parcial, tenta reverter (best-effort) os já movidos. O caller nunca deve alterar a base de dados a não ser que `ok` seja `True`.

5.2 `r2_service.py`

`app/services/r2_service.py` · 54 linhas

Wrapper fino sobre `boto3` para o Cloudflare R2 (API compatível com S3): `upload_bytes`, `copy_object` (propaga exceções — importante para o "all-or-nothing" do mover pastas), `delete_object` (engole exceções — apagar é best-effort), `presigned_url` (para downloads/previews sem expor credenciais).

5.3 `creditos_service.py`

`app/services/creditos_service.py`

Único ponto de entrada para alterar o saldo de créditos de um utilizador. Cada ação tem a sua função (`dar_creditos_upload`, `cobrar_creditos_download`, `creditos_ao_registar`, `dar_creditos_comunidade_post`, `dar_creditos_comunidade_resposta`), lê o valor de um config key (`CREDITOS_POR...`) e **faz o próprio commit** — ao contrário da maioria dos outros serviços. Isto significa que chamar uma função de créditos "fecha" a transação atual; código chamado depois começa uma nova.

5.4 `notificacoes_service.py`

`app/services/notificacoes_service.py`

`criar_notificacao()` é o primitivo de baixo nível (add, sem commit). As funções de alto nível (`notificar_aprovacao`, `notificar_rejeicao`, `notificar_moderadores_novo_material`) constroem o texto e chamam-na. A Comunidade (Fase 4) não tem funções de alto nível próprias — chama `criar_notificacao()` diretamente na rota.

5.5 `tokens_service.py`

`app/services/tokens_service.py`

Wrapper sobre `itsdangerous.URLSafeTimedSerializer`. Um *salt* por finalidade (`SALT_CONFIRMACAO`, `SALT_RECUPERACAO`, `SALT_CONVITE`, `SALT_MARKETING_CANCELAR`) garante que um token gerado para um fluxo não pode ser reutilizado noutro, mesmo que a assinatura use a mesma `SECRET_KEY`. `verificar_token()` recebe um `max_age` em segundos — o de cancelamento de marketing usa 10 anos (não deve expirar nunca).

5.6 `mail_service.py`

`app/services/mail_service.py` · 203 linhas

SMTP puro via `smtplib` — sem Flask-Mail. `enviar_email()` falha em silêncio (devolve `False`, regista um log) se as variáveis `MAIL_*` não estiverem configuradas — útil em desenvolvimento, mas significa que um erro de configuração em produção não gera exceção visível, só uma linha no log.

`enviar_email_marketing()` (Fase 4) é a variante para campanhas em massa: envia sempre multipart com uma versão texto simples gerada automaticamente (`html_para_texto_simples()`, um stripper de tags por regex) antes da versão HTML, e acrescenta os cabeçalhos `List-Unsubscribe/List-Unsubscribe-Post` (RFC 8058) e `Precedence: bulk` — tudo pensado para reduzir a probabilidade de o remetente ser marcado como spam.

5.7 `atividade_service.py`

`app/services/atividade_service.py`

`registar_atividade(evento, utilizador_id, alvo_tipo, alvo_id, detalhes)` — o `detalhes` pode ser um dict/list Python, que é automaticamente serializado para JSON antes de guardar. `atividade_recente()` e `contagem_por_evento()` alimentam o painel de KPI.

5.8 `marketing_service.py` Fase 4

`app/services/marketing_service.py` · 183 linhas

O serviço mais complexo do projeto. Envia campanhas de email em massa numa `threading.Thread` própria (sem Celery/Redis — decisão deliberada para não acrescentar infraestrutura nova a um projeto que ainda não precisa dela). Proteções contra bloqueio pelo servidor SMTP:

- **Throttle** — `MARKETING_INTERVALO_SEGUNDOS` de pausa entre cada envio
- **Limite diário** — `MARKETING_LIMITE_DIARIO` ; ao atingir, a campanha passa a `pausada_limite_diario` e retoma automaticamente no dia seguinte (ou quando o admin clicar "Continuar")
- **Retomável** — como cada destinatário é uma linha em `CampanhaEmailDestinatario` , um reinício do servidor a meio do envio não perde nem duplica progresso

Detalhe de arquitetura relevante para debugging: a thread de envio corre fora do contexto de um pedido HTTP, por isso não tem acesso a `request` . Para construir URLs absolutas corretas (o link de cancelamento de subscrição), a rota que inicia o envio captura `request.url_root` antes de arrancar a thread e passa-o como argumento; dentro da thread, cada envio usa `with app.test_request_context(base_url=base_url):` para poder chamar `url_for(..., _external=True)` com o domínio certo. Sem isto, o Flask levanta `RuntimeError: Unable to build URLs outside an active request` — este bug foi encontrado e corrigido durante o desenvolvimento da Fase 4.

Ordem de verificação no ciclo principal — **importante:** o código verifica primeiro se ainda há destinatários pendentes, e só *depois* verifica o limite diário. Se fosse ao contrário, uma campanha cujo número de destinatários coincidissem exatamente com o limite diário ficaria presa para sempre em `pausada_limite_diario` mesmo já não tendo mais ninguém a quem enviar — outro bug encontrado e corrigido durante o desenvolvimento.

5.9 `comunidade_service.py` Fase 4

`app/services/comunidade_service.py`

`aplicar_voto(utilizador, alvo_tipo, alvo_id, alvo_obj, valor)` — lógica de voto partilhada entre posts e respostas. Três casos: sem voto anterior → cria linha, aplica delta completo; mesmo valor que já tinha → remove o voto (toggle off), delta negativo; valor oposto → atualiza a linha, delta de magnitude 2. O `alvo_obj.votos_score` é atualizado diretamente em memória pelo delta — evita um `SELECT COUNT/SUM` a cada voto. `feed_query()` constrói a query base do feed (posts fixados sempre primeiro).

6. Utilitários — `app/utils/`

6.1 `honeypot.py`

`app/utils/honeypot.py`

`honeypot_preenchido(campo="website")` — verifica se o campo-armadilha invisível (só bots o preenchem) veio preenchido no POST. Usado em conjunto com o parcial `partials/honeypot.html` em todos os formulários públicos sem login prévio.

6.2 `validacao.py`

`app/utils/validacao.py`

`senha_forte(password)` — mínimo 8 caracteres, exige maiúscula, minúscula e dígito (via regex). Introduzido na Fase 1; qualquer fluxo de definição de senha no projeto deve chamar esta função.

7. Templates — `app/templates/`

Jinja2, ~70 ficheiros. `base.html` é o layout raiz (navbar, toasts de flash messages, banner, footer, scripts). Praticamente todos os outros templates fazem `{% extends "base.html" %}`, exceto os de `email/` (HTML autónomo, para compatibilidade com clientes de email) e os de `admin/` (estendem `admin/base_admin.html`, que por sua vez estende `base.html` e acrescenta a barra de navegação lateral do painel).

Diretório	Conteúdo
<code>admin/</code>	Todo o painel administrativo — um template por secção listada em 4.5, mais <code>base_admin.html</code> (define a lista <code>admin_links</code> usada para gerar a navegação)
<code>auth/</code>	Login, registo, recuperação de senha, email não confirmado
<code>comunidade/</code> Fase 4	<code>feed.html</code> , <code>detalhe.html</code> , <code>post_form.html</code> , e o parcial <code>_post_card.html</code> (reutilizado dentro do feed)
<code>dashboard/</code>	<code>index.html</code> (área privada) e <code>perfil.html</code>
<code>email/</code>	Um HTML autónomo por tipo de email — todos seguem o mesmo estilo visual de marca (gradiente azul-marinho/laranja no cabeçalho). <code>marketing_base.html</code> (Fase 4) é o wrapper que envolve o corpo escrito pelo admin numa campanha
<code>errors/</code>	403, 404, 429, 500
<code>main/</code>	Páginas institucionais e formulários públicos (sobre, suporte, candidatar-se, anunciar, etc.)
<code>materials/</code>	<code>listar.html</code> (pesquisa + filtros), <code>detalhe.html</code> , <code>submeter.html</code> , <code>pastas.html</code> (navegação por pastas, Fase 3), e o parcial <code>_card.html</code> extraído na Fase 3 para ser partilhado entre <code>listar.html</code> e <code>pastas.html</code>
<code>notificacoes/</code>	<code>index.html</code>
<code>partials/</code>	<code>honeypot.html</code> , <code>banner.html</code> (o banner de anúncio — desde a Fase 4 só é incluído por <code>comunidade/feed.html</code>)

Este documento não descreve cada um dos ~70 ficheiros HTML individualmente — a maioria é apresentação direta sem lógica própria além de `{% if %}` / `{% for %}`. Para os padrões de UI reutilizáveis (cards, paginação, filtros de sidebar, modais de denúncia), o código mais representativo para copiar como referência é `materials/listar.html` e `materials/_card.html`.

8. Estáticos — `app/static/`

`css/ponto24.css` é o design system próprio do projeto (cores de marca, cartões `.p24-card`, tabelas `.p24-table`, sidebar de filtros `.p24-sidebar` / `.p24-filter-item`, breadcrumbs, badges). Construído sobre Bootstrap 5, com media queries próprias `max-width` (desktop-first nalgumas partes mais antigas — inconsistência conhecida com a regra "mobile-first" adotada a partir da Fase 1; componentes novos seguem o grid naturalmente mobile-first do Bootstrap em vez de reescrever o CSS legado). `img/` guarda o logótipo. `uploads/` só é usado quando não há `R2_ENDPOINT` configurado — espelha exatamente a estrutura de pastas usada como `subfolder` em `guardar_ficheiro()`.

9. Configuração — config/

config/settings.py

Três classes: `DevelopmentConfig`, `ProductionConfig`, `TestingConfig`, todas herdando de `Config` (valores compartilhados). A mais relevante para produção: `ProductionConfig.init_app()` tem um `assert` que impede a app de arrancar se a `SECRET_KEY` ainda for o valor de exemplo inseguro. `config/__init__.py` só reexporta o dicionário `config` (mapeia o nome do ambiente para a classe) usado por `create_app()`.

Grupo	Chaves
Segurança	<code>SECRET_KEY</code> , <code>SESSION_COOKIE_*</code>
Upload	<code>UPLOAD_FOLDER</code> , <code>MAX_CONTENT_LENGTH</code> , <code>ALLOWED_EXTENSIONS</code>
Créditos	<code>CREDITOS_INICIAIS</code> , <code>CREDITOS_POR_UPLOAD_APROVADO</code> , <code>CREDITOS_POR_DOWNLOAD</code> , <code>CREDITOS_POR_POST_COMUNIDADE</code> , <code>CREDITOS_POR_RESPOSTA_COMUNIDADE</code>
Marketing Fase 4	<code>MARKETING_INTERVALO_SEGUNDOS</code> , <code>MARKETING_LIMITE_DIARIO</code>

10. Migrações — migrations/

Geridas por Flask-Migrate/Alembic. `env.py` é o ficheiro de arranque do Alembic (gerado automaticamente pelo `flask db init`, raramente precisa de edição manual). Cada ficheiro em `versions/` corresponde a um `flask db migrate`, em ordem cronológica através de `down_revision`.

Armadilha conhecida do SQLite em modo batch: ao acrescentar uma `ForeignKeyConstraint` a uma tabela já existente via `batch_alter_table(...).create_foreign_key(None, ...)`, o Alembic autogerado usa `None` como nome da constraint — e o SQLite, ao recriar a tabela em modo batch, exige que a constraint tenha um nome. Isto já aconteceu duas vezes neste projeto (migração das pastas e, de forma preventiva, também nomeada explicitamente nas seguintes). Se o `flask db upgrade` falhar com `ValueError: Constraint must have a name`, é isto — edita a migração e dá um nome explícito à constraint, tanto no `upgrade()` como no `downgrade()`.

Colunas NOT NULL novas em tabelas com dados: acrescentar uma coluna `nullable=False` sem `server_default` a uma tabela que já tem linhas falha, porque a base de dados não sabe que valor dar às linhas existentes (aconteceu com `aceita_marketing` na Fase 4). Solução: acrescentar `server_default=sa.true()` (ou o valor apropriado) na migração, e opcionalmente remover o default a seguir com um segundo `alter_column`.

11. Ficheiros de raiz

11.1 `manage.py`

`manage.py` · 354 linhas

Comandos CLI registados via `@app.cli.command(...)` (Click, integrado no Flask): `seed-db`, `criar-admin`, `testar-email`, e `importar-lote` — este último é o mais elaborado: percorre uma árvore de pastas `INSTITUICAO/DISCIPLINA/CATEGORIA/ficheiro`, faz correspondência aproximada do nome da categoria (`_resolver_categoria`), deteta duplicados por hash, numera automaticamente títulos repetidos ("Disciplina — Parte 2"), e comita em lotes de 100. Suporta `--dry-run` para pré-visualizar sem alterar nada — recomenda-se sempre correr assim primeiro.

11.2 `run.py` / `wsgi.py`

`run.py`, `wsgi.py`

`run.py` — entrada de desenvolvimento (`app.run(debug=True)`). `wsgi.py` — entrada de produção, o objeto `app` que o Gunicorn importa (`gunicorn -w 4 -b 0.0.0.0:8000 wsgi:app`).

12. Convenções e padrões de arquitetura

Uma lista consolidada dos padrões que se repetem por todo o código — reconhecê-los acelera muito a leitura de qualquer ficheiro novo do projeto.

"Add mas não commit" nos serviços

Funções como `criar_notificacao()` e `registar_atividade()` só fazem `db.session.add()`. Quem chamou decide quando fazer `db.session.commit()`, normalmente depois de várias operações relacionadas, para que tudo aconteça numa única transação atômica. Exceção: os serviços de créditos fazem o próprio commit (ver 5.3).

Referências polimórficas por convenção (`alvo_tipo` + `alvo_id`)

Usado em `AtividadeLog`, `ComunidadeVoto` e `ComunidadeRelatorio`. Uma tabela consegue apontar para linhas de tabelas diferentes sem uma foreign key real — o preço é que a base de dados não valida `alvo_id`; é o código que tem de tratar o caso de o alvo já não existir (ver `_resolver_alvo_comunidade()` em `admin.py`, que devolve `None` graciosamente).

Caminho materializado (`Pasta.caminho`)

Ver secção 3.3. Só é seguro porque não há reparenting de pastas no produto atual.

Operações all-or-nothing com rollback best-effort

`mover_grupo_para_pasta()` (secção 5.1) é o exemplo canónico: mover vários ficheiros fisicamente é uma operação que pode falhar a meio, e a base de dados não deve nunca refletir um estado que não corresponde à realidade do disco/R2.

Storage local vs. R2 por variável de ambiente

Praticamente todo o código que toca em ficheiros testa `os.environ.get("R2_ENDPOINT")` (diretamente ou via `_usar_r2()`) e ramifica. Não existe uma interface abstrata única — o padrão se-então repete-se em vários sítios (`upload_service.py`, `app/__init__.py` nos helpers de URL, `admin.py` nos anúncios). Ao acrescentar uma nova funcionalidade com ficheiros, replicar este padrão em vez de inventar um novo.

Eventos/tipos como strings-constantes de classe, não enums de base de dados

`Material.STATUS_*`, `Notificacao.TIPO_*`, `AtividadeLog.EVENTO_*`, `CampanhaEmail.ESTADO_*` — todos strings simples guardadas em colunas `String`, não enums nativos do SGBD. Vantagem: acrescentar um novo valor nunca exige migração. Desvantagem: nada impede um valor inválido ser escrito por engano — a validação é sempre responsabilidade do código da rota.

Extração de parciais para eliminar duplicação de template

`materials/_card.html` (Fase 3) e `comunidade/_post_card.html` (Fase 4) seguem o mesmo princípio: quando o mesmo bloco de HTML apareceria em duas páginas (lista normal + navegação por pastas; feed +

potencial reaproveitamento futuro), extrai-se para um parcial incluído com `{% include %}`, que herda o contexto Jinja da página que o inclui.

13. Modelo de dados — tabelas e relações

Resumo de todas as tabelas da base de dados e as suas relações principais (FK). Para os campos completos, ver a secção 3.

Tabela	Relaciona-se com	Fase
users	—	Base
categorias	—	Base
materiais	users (autor, moderador), categorias, pastas	Base / F3
favoritos	users, materiais	Base
avaliacoes	users, materiais	Base
pastas	pastas (auto-referência), users (criador)	F3
notificacoes	users	Base
configuracoes	—	Base
lista_espera	—	Base
relatorios_materiais	materiais, users	Base
anuncios	—	Base
candidaturas	—	Base
atividade_log	users (opcional); alvo_tipo / alvo_id sem FK	F2
pesquisa_log	users (opcional)	Base
campanhas_email	users (criador)	F4
campanha_email_destinatarios	campanhas_email, users	F4
comunidade_posts	users (autor)	F4
comunidade_post_imgs	comunidade_posts	F4
comunidade_respostas	comunidade_posts, users	F4
comunidade_votos	users; alvo_tipo / alvo_id sem FK	F4
comunidade_relatorios	users (opcional); alvo_tipo / alvo_id sem FK	F4

14. Fluxos ponta-a-ponta importantes

14.1 Submissão e aprovação de um material

1. `materiais.submeter()` valida, guarda ficheiro(s) via `guardar_ficheiro()`, deteta duplicados por hash, cria uma linha `Material` por ficheiro (`STATUS_PENDENTE`), regista atividade, comita
2. `notificar_moderadores_novo_material()` cria notificação in-app + email para todos os admins/moderadores
3. Um moderador vê a fila em `admin.moderacao()` ou revisa individualmente em `admin.rever_material()`
4. `admin.aprovar()`: muda o status, chama `dar_creditos_upload()` (commit próprio), `notificar_aprovacao()`, `registar_atividade()`, comita, e só depois envia `email_material_aprovado()`
5. O material passa a aparecer em `materiais.listar()` e na navegação por pastas

14.2 Envio de uma campanha de marketing

1. Admin cria a campanha (`admin.criar_campanha()`) — fica `rascunho`
2. Ao clicar "Enviar" (`admin.enviar_campanha()`): a rota captura `request.url_root`, chama `marketing_service.iniciar_envio()`, que popula `CampanhaEmailDestinatario` (uma vez só, público filtrado por `aceita_marketing=True` + segmento escolhido) e arranca a thread
3. A thread processa destinatários `pendente` um a um, respeitando o throttle e o limite diário, atualizando o progresso a cada envio
4. A página de detalhe (`admin/marketing_detalhe.html`) faz polling a `/admin/marketing/<id>/estado` a cada poucos segundos para mostrar a barra de progresso ao vivo
5. Se atingir o limite diário com destinatários ainda pendentes, a campanha fica `pausada_limite_diario`; o admin pode clicar "Continuar" nesse dia ou no seguinte

14.3 Votar num post da Comunidade

1. Utilizador clica na seta de upvote/downvote (formulário POST simples, sem JavaScript/AJAX)
2. `comunidade.votar_post()` / `votar_resposta()` chama o helper partilhado `_votar()`, que bloqueia auto-voto e delega a `comunidade_service.aplicar_voto()`
3. `aplicar_voto()` decide entre criar/remover/trocar o voto e ajusta `votos_score` diretamente
4. Redireciona de volta para `request.referrer` — se o voto veio do feed, o utilizador continua no feed, não é forçado a ir para o detalhe do post

15. Guia rápido de debugging

Sintoma	Onde procurar primeiro
Algo estranho em <i>todas</i> as páginas (redirect, variável em falta, cabeçalho)	<code>app/__init__.py</code> — hooks e context processor (secção 2)
Email não chega	Confirma <code>MAIL_*</code> no <code>.env</code> ; sem eles, <code>enviar_email()</code> falha em silêncio e só regista um log em nível <code>debug</code>
Upload falha só em produção, não localmente	Provavelmente falta configurar <code>R2_*</code> — em disco local, um deploy sem volume persistente perde os ficheiros a cada release
<code>flask db upgrade</code> falha com "Constraint must have a name"	Ver nota da secção 10 — SQLite batch mode
Campanha de marketing presa em "pausada" sem nunca terminar	Ver secção 5.8 — confirmar se o bug da ordem de verificação (corrigido na Fase 4) não reapareceu em código novo copiado deste padrão
Link num email de marketing aponta para <code>localhost</code>	A thread de envio não tem <code>request</code> — confirmar que <code>base_url</code> está a ser passado corretamente desde a rota até <code>processar_fila()</code>
403 inesperado numa rota <code>admin</code>	Confirmar qual decorador está a ser usado — <code>admin_required</code> é mais restritivo que <code>moderador_required</code> ; muitas secções da Fase 4 (Comunidade) usam o segundo de propósito
Contagem denormalizada (<code>respostas_count</code> , <code>votos_score</code>) dessincronizada da realidade	Só é incrementada/decrementada manualmente no código (criar/eliminar resposta ou voto) — nunca há um recálculo agendado; se um caminho de código novo esquecer de o fazer, a contagem fica errada silenciosamente
Quero encontrar todas as ações de um utilizador específico	<code>AtividadeLog.query.filter_by(utilizador_id=...).order_by(AtividadeLog.criado_em.desc())</code> — cobre praticamente todas as ações relevantes desde a Fase 2